

BUỔI 10 - Tích hợp Toàn hệ thống Microservices & Hướng dẫn Tự học Docker Compose

Điều kiện tiên quyết: đã hoàn thành Buổi 9 - luồng Đăng nhập → Xem môn học → Đăng ký → Huỷ đăng ký hoạt động đầy đủ qua Gateway, có Toast xử lý lỗi nghiệp vụ.
Đây là buổi tổng kết - không viết tính năng mới, mà kiểm chứng lại toàn bộ 9 buổi trước đã ráp đúng thành 1 hệ thống thống nhất chưa, đồng thời khép lại phần dạy trên lớp.

0. Mục tiêu buổi học

Sau buổi học này, sinh viên phải:

- Tự khởi động đúng thứ tự cả 5 thành phần (auth-service, course-service, registration-service, api-gateway, crs-frontend) và xác nhận cả hệ thống hoạt động như một khối thống nhất.
- Chạy đúng kịch bản đầu-cuối (end-to-end): Đăng nhập → Xem môn học → Đăng ký → Huỷ đăng ký → Đăng xuất, đối chiếu JSON và Status Code ở từng bước qua DevTools.
- Chủ động tạo ra lỗi (tắt course-service) để quan sát trực tiếp cách registration-service xử lý và cách lỗi đó lan truyền lên tận Frontend - không né tránh tình huống lỗi như một bài học thực tế.
- Rà soát lại toàn bộ lớp bảo mật đã xây (JWT, API Key, Role) bằng 1 lượt test tổng hợp.
- Gắn tag v1.0 cho bản hoàn chỉnh trên Git.
- Hiểu ở mức khái niệm vì sao Docker Compose là bước tiếp theo hợp lý cho hệ thống 5 thành phần này (phần tự học, không bắt buộc).

A. Khởi động toàn bộ hệ thống

A.1. Thứ tự khởi động khuyến nghị

Không bắt buộc theo đúng thứ tự tuyệt đối (Spring Boot service nào chạy trước đều tự chờ được), nhưng khuyến nghị thứ tự sau để dễ quan sát log khi có lỗi:

1. auth-service (8081)
2. course-service (8082)
3. registration-service (8083)
4. api-gateway (8080)
5. crs-frontend (npm run dev, thường ở 5173)

A.2. Bảng kiểm tra nhanh trước khi bắt đầu test

Thành phần	Cổng	Lệnh / cách kiểm tra nhanh	Kỳ vọng
auth-service	8081	GET http://localhost:8081/auth/login (gọi sai method, chỉ cần server phản hồi)	Nhận được phản hồi (kể cả lỗi 405) nghĩa là service đã sống
course-service	8082	GET http://localhost:8082/courses	200, JSON danh sách môn học
registration-service	8083	GET http://localhost:8083/registrations/my (không token)	401/403 - nghĩa là service sống và đang áp bảo mật đúng
api-gateway	8080	GET http://localhost:8080/api/courses	200, JSON giống hết gọi thẳng

			8082/courses
crs-frontend	5173	Mở trình duyệt	Trang /courses hiển thị đúng danh sách

GHI CHÚ: Nếu bước nào ở bảng trên sai, dừng lại xử lý ngay trước khi sang mục B - chạy kịch bản đầu-cuối trên nền tảng chưa ổn định sẽ gây khó khăn khi xác định lỗi thực sự nằm ở đâu.

B. Kịch bản đầu-cuối (end-to-end)

Mở DevTools (tab Network) trước khi bắt đầu, để quan sát từng request thật sự đi qua Gateway.

Bước	Thao tác trên giao diện	Request thực tế (DevTools)	Kỳ vọng
1	Vào /login, đăng nhập student1/student123	POST /api/auth/login	200, response có token, userId, role: STUDENT
2	Tự động chuyển tới /courses	GET /api/courses?page=0&size=10	200, không kèm Authorization (route public)
3	Gõ từ khoá tìm kiếm bất kỳ	GET /api/courses?keyword=...&page=0	200, danh sách lọc đúng, quay về trang 1
4	Vào /register-course, bấm "Đăng ký" 1 môn còn chỗ	POST /api/registrations	201, kèm Authorization: Bearer <token>; đối chiếu soChoConLai giảm 1 ở lần GET /api/courses tiếp theo
5	Vào /my-registrations	GET /api/registrations/my rồi nhiều GET /api/courses/{id} ghép tên	200 tất cả, danh sách hiện đúng môn vừa đăng ký
6	Bấm "Huỷ đăng ký"	DELETE /api/registrations/{id}	200, dòng biến mất, soChoConLai của môn đó tăng lại 1
7	Bấm "Đăng xuất"	(không có request mạng, chỉ xoá localStorage và chuyển hướng)	Quay về /login, thử gõ tay /register-course → bị đá về /login

GHI CHÚ QUAN TRỌNG: Ở bước 5, sinh viên sẽ thấy rõ trên Network tab nhiều request GET /api/courses/{id} liên tiếp - đây chính là kỹ thuật API composition đã cài đặt ở Buổi 9. Đây là dịp tốt để giảng viên đặt câu hỏi mở: "Nếu 1 sinh viên đăng ký 20 môn, trang này sẽ gọi bao nhiêu request?" → dẫn tới thảo luận ngắn về đánh đổi giữa đơn giản (Frontend tự ghép) và hiệu năng (nên có 1 API tổng hợp ở Backend) - đúng là hướng mở rộng hợp lý nếu sinh viên làm đồ án riêng sau khoá.

C. Kịch bản lỗi: tắt course-service

C.1. Thực hiện

- Trong IntelliJ, dừng hẳn tiến trình course-service (nút Stop, không phải chỉ đóng cửa sổ).
- Trên Frontend đang mở sẵn (không F5), thử vào /register-course và bấm "Đăng ký" một môn bất kỳ.

C.2. Quan sát và giải thích

- Request POST /api/registrations gửi tới Gateway, Gateway forward đúng tới registration-service (8083) - service này vẫn đang chạy bình thường.
- registration-service cố gọi CourseClient.reserveSeat() (viết từ Buổi 3, mục C.6) sang http://localhost:8082/internal/courses/.../reserve-seat, nhưng course-service đã tắt.

- RestTemplate sẽ ném ResourceAccessException (kết nối bị từ chối) sau một khoảng thời gian nhất định (mặc định không có timeout tường minh - đây cũng là điểm cần nêu ra như hạn chế đáng lưu ý).
- CourseClient đã bắt đúng exception này từ Buổi 3 và ném lại IllegalStateException("Không thể kết nối tới course-service...").
- GlobalExceptionHandler của registration-service (Buổi 3, mục C.9) bắt IllegalStateException, trả về JSON {"message": "Không thể kết nối tới course-service, vui lòng thử lại sau"} với status 409.
- Toast đỏ trên Frontend (Buổi 9) hiển thị đúng message này - không hề crash, không hề treo vô thời hạn.

C.3. Thảo luận trên lớp

Câu hỏi cho sinh viên: "Đây là ví dụ cụ thể của rủi ro gì trong kiến trúc microservices mà monolith không gặp phải?"

Gợi ý:

- Single point of failure phân tán: một service con sập không làm sập toàn hệ thống, nhưng làm hỏng đúng những tính năng phụ thuộc vào nó - ở đây là toàn bộ luồng đăng ký/hủy đăng ký, trong khi việc xem danh sách môn học (GET /api/courses gọi thẳng course-service) đã tắt nên cũng không hoạt động, còn đăng nhập (auth-service) vẫn hoạt động bình thường.
- Không có timeout tường minh ở RestTemplate mặc định là một rủi ro thực tế - nếu course-service không "chết hẳn" mà chỉ phản hồi rất chậm (treo), registration-service có thể bị giữ thread chờ rất lâu, ảnh hưởng dây chuyền tới toàn hệ thống. Đây gọi là hiệu ứng cascading failure. Hướng giải quyết chuẩn trong thực tế (circuit breaker - ví dụ Resilience4j) nằm ngoài phạm vi 10 buổi, có thể tự nghiên cứu mở rộng.
- Đối chiếu lại với thảo luận về giao dịch phân tán đã làm ở Buổi 3: nếu course-service sập đúng lúc đang xử lý dở dang, đây chính là kịch bản cụ thể hoá cho rủi ro lệch dữ liệu đã cảnh báo trước đó.

C.4. Khôi phục

Bật lại course-service, thử lại thao tác đăng ký - xác nhận hệ thống phục hồi hoàn toàn bình thường, không cần khởi động lại bất kỳ service nào khác.

D. Rà soát bảo mật tổng hợp

Chạy lại nhanh bằng test tổng hợp (không cần làm hết mọi trường hợp đã test rải rác ở Buổi 4/7/8/9, chỉ chạy đại diện):

#	Kịch bản	Gọi	Kỳ vọng
1	Không có token, gọi API cần đăng nhập	POST /api/registrations (không header)	401, chặn tại Gateway
2	Có token STUDENT, gọi API chỉ ADMIN	POST /api/courses với token student1	403, chặn tại course-service
3	Có token ADMIN, gọi đúng API ADMIN	POST /api/courses với token admin	201
4	Token giả mạo (sửa 1 ký tự bất kỳ trong chuỗi JWT)	Bất kỳ API cần đăng nhập nào	401 - chữ ký không hợp lệ, JwtAuthFilter không set được Authentication
5	Route đối tác không có API Key	GET /api/public/courses (không header X-API-KEY)	403

6	Route đối tác có API Key đúng	GET /api/public/courses với X-API-KEY đúng	200
7	Gọi thẳng /internal/courses/{id}/reserve-seat từ Postman tới cổng 8082 (bỏ qua Gateway)	-	Vẫn thành công (vì permitAll() theo thiết kế Buổi 4) - nhắc lại đây là giới hạn đã biết: bảo mật cho /internal/** hiện dựa vào việc Gateway không định tuyến ra ngoài, không phải dựa vào xác thực thật sự; nếu ai đó truy cập được trực tiếp vào mạng nội bộ (localhost trong môi trường học), vẫn gọi được. Ghi nhận như hướng cải thiện nếu làm đồ án riêng (ví dụ: chỉ cho phép IP nội bộ, hoặc dùng secret riêng giữa các service)

E. Checklist hoàn thiện dự án CRS Microservices

Rà soát lần cuối, đối chiếu với toàn bộ 9 buổi đã làm:

- ☐ 4 service chạy độc lập, đúng cổng đã thiết kế từ Buổi 1 (8080/8081/8082/8083), mỗi service (trừ Gateway) có DB riêng.
- ☐ registration-service gọi thành công sang course-service để reserve/release chỗ (Buổi 3).
- ☐ api-gateway định tuyến đúng cả 3 service, là điểm vào duy nhất cho Frontend (Buổi 4).
- ☐ CORS chỉ cấu hình 1 nơi - tại Gateway (Buổi 4, Buổi 5).
- ☐ JWT được xác thực đúng ở từng service nhận request; API Key hoạt động đúng cho route đối tác (Buổi 4, mục D ở trên).
- ☐ Phân quyền ADMIN/STUDENT đúng cả Backend (SecurityConfig, Buổi 4) lẫn Frontend (ProtectedRoute, Role-Based UI, Buổi 8).
- ☐ Giao diện xử lý đủ Loading/Success/Empty/Error, kể cả lỗi lan truyền từ service khác (Buổi 6, Buổi 9, mục C ở trên).
- ☐ Git có lịch sử commit rõ theo từng buổi, cho cả 4 service + frontend.

LƯU Ý: Nếu bất kỳ dòng nào chưa đạt, đây là thời điểm cuối cùng trong 10 buổi để quay lại buổi tương ứng khắc phục trước khi gắn tag.

F. Git commit & gắn tag v1.0

```
cd crs-microservices
git add .
git commit -m "chore: final integration review for v1.0"
git tag -a v1.0 -m "CRS Microservices - bản hoàn chỉnh sau 10 buổi thực hành"
git push
git push origin v1.0
```

G. Hướng dẫn tự học Docker Compose (không bắt buộc)

G.1. Vì sao đến lúc này Docker Compose mới thật sự có ý nghĩa?

Trong suốt 9 buổi trước, hệ thống chỉ có 1-2 thành phần mới mỗi buổi - tự chạy tay từng cửa sổ IntelliJ là đủ. Đến hết Buổi 10, hệ thống đã có **5 thành phần triển khai độc lập** (4 service Spring Boot + 1 Frontend) cộng **3 database MySQL riêng biệt** - bật tay từng cái mỗi lần ngồi vào máy bắt đầu trở nên bất tiện thật sự. Đây đúng là lúc Docker Compose phát huy giá trị.

G.2. Ý tưởng: đổi localhost thành tên container

Toàn bộ cấu hình từ Buổi 1 đến Buổi 9 đều dùng localhost để các service gọi nhau (course-service.base-url=http://localhost:8082, JDBC URL jdbc:mysql://localhost:3306/...). Khi chạy trong Docker, mỗi container có địa chỉ mạng riêng - Docker Compose tạo ra 1 mạng nội bộ nơi các container gọi nhau **bằng tên service khai báo trong file YAML**, thay vì localhost.

Spring Boot hỗ trợ sẵn việc **ghi đè cấu hình bằng biến môi trường** (không cần sửa file application.properties đã viết suốt 9 buổi) - ví dụ biến môi trường SPRING_DATASOURCE_URL sẽ tự động ghi đè spring.datasource.url.

G.3. Cấu trúc docker-compose.yml mẫu

```
# path: crs-microservices/docker-compose.yml
# purpose: định nghĩa 5 service + 3 MySQL container, dùng mạng nội bộ Docker để gọi nhau
# qua TEN SERVICE thay vì localhost. Đây là file THAM KHẢO cho sinh viên tu học,
# không bắt buộc phải chạy dung trong 10 buổi trên lớp.

version: '3.8'

services:
  mysql-auth:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: root123
      MYSQL_DATABASE: auth_db
    ports: ["3307:3306"]

  mysql-course:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: root123
      MYSQL_DATABASE: course_db
    ports: ["3308:3306"]

  mysql-registration:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: root123
      MYSQL_DATABASE: registration_db
    ports: ["3309:3306"]

  auth-service:
    build: ./auth-service
    depends_on: [mysql-auth]
```

```

environment:
  SPRING_DATASOURCE_URL: jdbc:mysql://mysql-auth:3306/auth_db
  SPRING_DATASOURCE_USERNAME: root
  SPRING_DATASOURCE_PASSWORD: root123
  JWT_SECRET: CRS-Microservices-Secret-Key-Nam-3-Hoc-Ky-2026-Doi-Trong-Thuc-Te
ports: ["8081:8081"]

course-service:
  build: ./course-service
  depends_on: [mysql-course]
  environment:
    SPRING_DATASOURCE_URL: jdbc:mysql://mysql-course:3306/course_db
    SPRING_DATASOURCE_USERNAME: root
    SPRING_DATASOURCE_PASSWORD: root123
    JWT_SECRET: CRS-Microservices-Secret-Key-Nam-3-Hoc-Ky-2026-Doi-Trong-Thuc-Te
ports: ["8082:8082"]

registration-service:
  build: ./registration-service
  depends_on: [mysql-registration, course-service]
  environment:
    SPRING_DATASOURCE_URL: jdbc:mysql://mysql-registration:3306/registration_db
    SPRING_DATASOURCE_USERNAME: root
    SPRING_DATASOURCE_PASSWORD: root123
    JWT_SECRET: CRS-Microservices-Secret-Key-Nam-3-Hoc-Ky-2026-Doi-Trong-Thuc-Te
    # DIEM MAU CHOT: doi localhost thanh ten service "course-service" khai bao o tren
    COURSE_SERVICE_BASE_URL: http://course-service:8082
ports: ["8083:8083"]

api-gateway:
  build: ./api-gateway
  depends_on: [auth-service, course-service, registration-service]
  environment:
    # Cac URL dinh tuyen trong application.yml cung can doi tuong tu sang ten service
    AUTH_SERVICE_URL: http://auth-service:8081
    COURSE_SERVICE_URL: http://course-service:8082
    REGISTRATION_SERVICE_URL: http://registration-service:8083
ports: ["8080:8080"]

```

LƯU Ý: để các biến môi trường như COURSE_SERVICE_BASE_URL hay AUTH_SERVICE_URL thật sự có tác dụng, cần sửa lại application.properties/application.yml từ dùng giá trị cứng sang dùng cú pháp \${TEN_BIEN:gia-tri-mac-dinh} - ví dụ course-service.base-url=\${COURSE_SERVICE_BASE_URL:http://localhost:8082}.

Cách viết này vẫn chạy đúng khi mở tay từng service qua IntelliJ như 10 buổi vừa học (dùng giá trị mặc định localhost), và tự động chuyển sang đúng địa chỉ Docker khi chạy qua docker-compose up.

Mỗi service (Dockerfile riêng cho từng service, build từ JDK 17 + Maven) cũng cần được viết, sinh viên tự tìm hiểu mẫu Dockerfile cho Spring Boot nếu muốn hoàn thiện phần này.

G.4. Gợi ý áp dụng cho Bài tập lớn

- Tự viết Dockerfile cho từng service (4 Dockerfile riêng biệt, đúng tinh thần "mỗi service triển khai độc lập" đã học suốt khóa).

- Sửa cấu hình theo mẫu biến môi trường ở mục G.3.
- Chạy thử docker-compose up --build, xác nhận lại đúng kịch bản đầu-cuối ở mục B của buổi này vẫn hoạt động khi chạy hoàn toàn trong Docker.
- Đây là điểm dừng hợp lý để bắt đầu tìm hiểu thêm: health check, restart policy, volume cho MySQL (tránh mất dữ liệu khi container restart).

Sản phẩm đầu ra cuối buổi

- [] Cả 5 thành phần chạy đồng thời ổn định, kịch bản đầu-cuối ở mục B chạy trôi chảy không lỗi.
- [] Đã tự tay tạo và quan sát được kịch bản lỗi ở mục C, giải thích được bằng lời khái niệm rủi ro liên quan (single point of failure phân tán, cascading failure).
- [] Rà soát bảo mật tổng hợp ở mục D đạt đủ 7 trường hợp.
- [] Checklist đầy đủ ở mục E đạt 100%.
- [] Git có tag v1.0.
- [] Hiểu ở mức khái niệm ý tưởng cốt lõi của Docker Compose (đổi localhost thành tên service) dù chưa bắt buộc tự triển khai.

Lời kết

Hệ thống CRS hoàn chỉnh theo kiến trúc microservices thu nhỏ đã chạy đúng trên localhost, đi qua đầy đủ các chủ đề quan trọng: thiết kế biên giới service, giao tiếp REST liên-service, giao dịch phân tán đơn giản hoá, API Gateway, JWT xuyên service, và một Frontend React hoàn chỉnh chỉ giao tiếp qua 1 điểm vào duy nhất.

KHUYẾN NGHỊ CUỐI: Tránh áp dụng microservices chỉ vì "nghe có vẻ hiện đại hơn" trong khi độ phức tạp vận hành (nhiều service, nhiều DB, giao tiếp mạng) không tương xứng với quy mô bài toán thật.